

Tarefa 4 — Memory-Bound vs Compute-Bound com OpenMP

Vinicius Barbosa Ventura Mergulhão

CPU: 13th Gen Intel Core i5-13420H (4 P-cores + 8 E-cores = 12 threads lógicos) - só pra lembrar

1. Programas implementados

Programa	Operação	Característica
<code>memory_bound.c</code>	$C[i] = A[i] + B[i]$ em 100M doubles (~2.4 GB trafegados)	Baixa intensidade aritmética — gargalo no barramento de memória
<code>compute_bound.c</code>	10.000 iterações de <code>sin + cos</code> por elemento	Alta intensidade aritmética — gargalo nas unidades de ponto flutuante (FPU)

Ambos paralelizados com `#pragma omp parallel for`.

2. Resultados

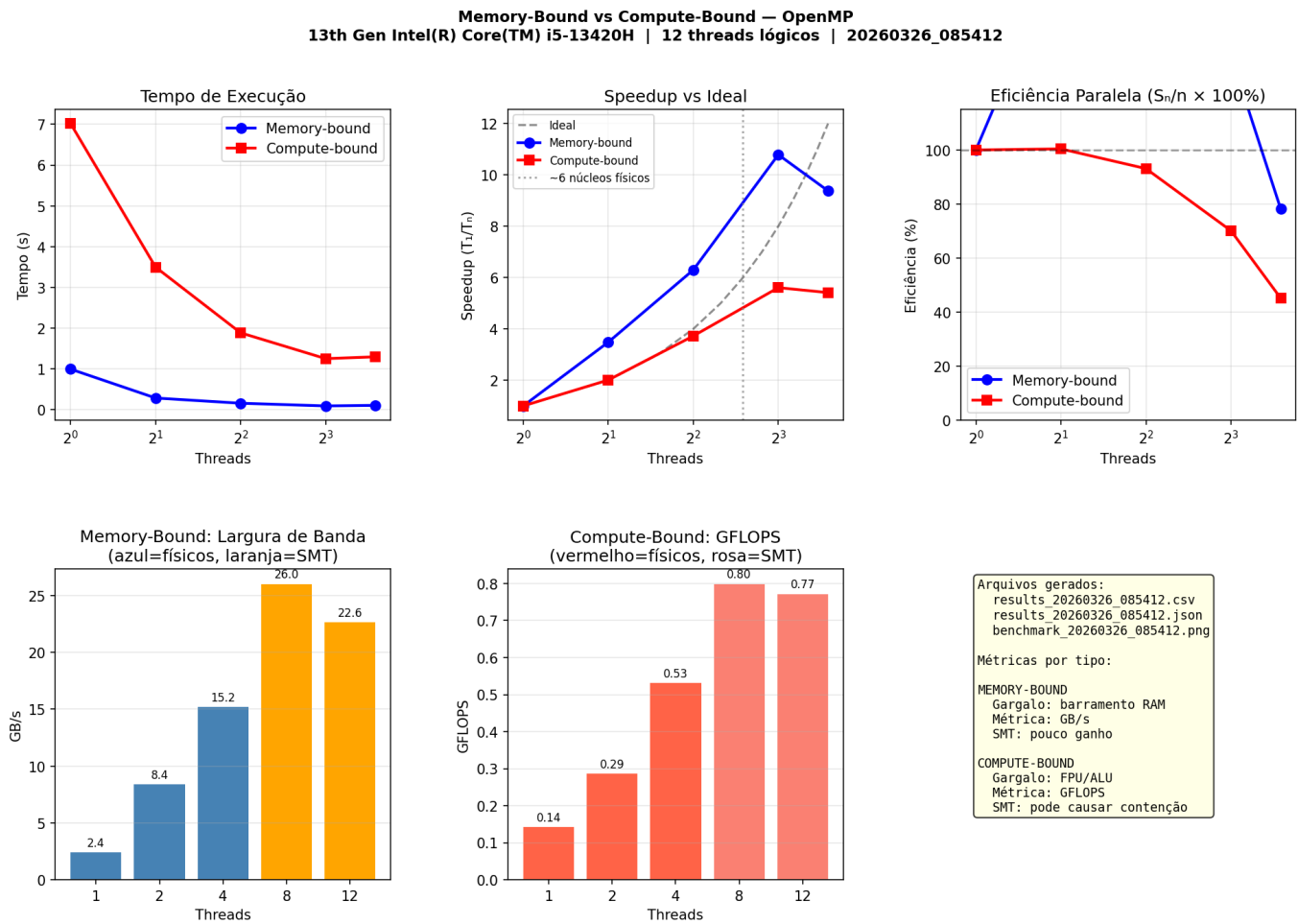
Memory-Bound

Threads	Tempo (s)	Speedup	GB/s	Eficiência
1	0.9950	1.00x	2.41	100%
2	0.2857	3.48x	8.40	174%
4	0.1580	6.30x	15.19	157%
8	0.0922	10.79x	26.02	135%
12	0.1061	9.38x	22.62	78%

Compute-Bound

Threads	Tempo (s)	Speedup	GFLOPS	Eficiência
1	7.0185	1.00x	0.142	100%
2	3.4956	2.01x	0.286	100%
4	1.8848	3.72x	0.531	93%
8	1.2518	5.61x	0.799	70%
12	1.2965	5.41x	0.771	45%

3. Gráficos gerados



O gráfico é dividido em 5 painéis:

Painel 1 — Tempo de Execução (linha azul e vermelha): Mostra como o tempo cai à medida que threads são adicionados. A linha do compute-bound (vermelha) parte de ~7s com 1 thread e cai de forma gradual. A linha do memory-bound (azul) parte de ~1s e cai muito mais rápido, chegando a ~0.09s com 8 threads — mas sobe levemente em 12. O eixo X usa escala logarítmica base 2.

Painel 2 — Speedup vs Ideal: Compara o speedup real com a linha ideal (pontilhada preta), onde **speedup = número de threads**. O memory-bound (azul) ultrapassa a linha ideal — speedup super-linear causado por Memory-Level Parallelism e pelo baseline pessimista com 1 thread. O compute-bound (vermelho) acompanha a linha ideal até ~2 threads e depois se distancia progressivamente, especialmente após os núcleos homogêneos serem esgotados. Em 12 threads, ambos mostram queda em relação ao pico (8 threads).

Painel 3 — Eficiência Paralela: Traduz o speedup em percentual por thread ($E = S/n \times 100\%$). O memory-bound começa acima de 100% (reflexo do super-linear) e cai para 78% em 12 threads. O compute-bound começa em 100% com 2 threads, cai para 93% em 4, 70% em 8 e despenca para 45% em 12 — confirmando que os threads extras passam a ser mais custo do que benefício.

Painel 4 — Largura de Banda em GB/s (barras azuis/laranja): Métrica central do memory-bound. As barras crescem de 2.4 GB/s (1 thread) até o pico de 26 GB/s (8 threads), onde o barramento de memória atinge sua capacidade máxima. A barra de 12 threads é menor (22.6 GB/s), evidenciando que adicionar threads além da saturação reduz a vazão por contenção.

Painel 5 — GFLOPS (barras vermelhas/rosa): Métrica central do compute-bound. Cresce de 0.14 GFLOPS (1 thread) até 0.80 GFLOPS (8 threads). A barra de 12 threads recua ligeiramente para 0.77 GFLOPS — os threads adicionais introduzem desbalanceamento e overhead de sincronização que superam o ganho de cálculo.

4. Análise

4.1 Memory-Bound — Speedup super-linear e saturação

A eficiência acima de 100% (super-linear) tem duas causas:

Baseline pessimista: O tempo de 1 thread teve variância alta (mín 0.30s, máx 2.96s). A mediana pegou uma execução ruim, provavelmente com efeito de cache frio ou agendamento desfavorável pelo SO.

Memory-Level Parallelism (MLP): Com 1 thread, há apenas uma fila de requisições ao controlador de memória. Com múltiplos threads, o controlador recebe requisições independentes e as serve em paralelo. Em memória dual-channel, threads em núcleos diferentes podem explorar ambos os canais simultaneamente — com 1 thread, apenas metade da banda disponível é utilizada.

Colapso em 12 threads: O barramento já estava saturado em ~26 GB/s com 8 threads. Os threads extras vão para núcleos de menor desempenho, gerando contenção no controlador de memória sem acrescentar vazão — o overhead de sincronização supera o ganho marginal.

Melhor ponto de operação: 8 threads — máxima largura de banda antes da saturação.

4.2 Compute-Bound — Escala linear depois queda

O compute-bound apresenta comportamento estável (variância mínima entre repetições) pois o trabalho é puramente computacional e independente entre iterações.

1 → 2 threads (100% eficiência): Carga perfeitamente balanceada entre dois núcleos equivalentes. Cada processador é 100% útil.

4 → 8 threads (queda de 93% → 70%): A partir de certo ponto, o escalonador distribui trabalho para núcleos de menor desempenho. O OpenMP divide o trabalho igualmente (`schedule(static)`), mas núcleos mais lentos terminam depois — a barreira implícita no final do `parallel for` faz todos esperarem o mais lento. Isso é **desbalanceamento de carga**.

8 → 12 threads (regressão de 5.61x → 5.41x): O tempo aumenta ligeiramente. Os threads extras adicionam overhead de sincronização sem agregar GFLOPS suficientes.

Melhor ponto de operação: 4 threads — escala a 93% de eficiência com carga homogênea.

5. Quadrante Memory-Bound × Compute-Bound

	Baixa Computação	Alta Computação
Poucos Dados	Subutilização dos recursos	COMPUTE-BOUND (programa 2)
Muitos Dados	MEMORY-BOUND (programa 1)	Bottlenecks em tudo

- **Memory-Bound:** hardware suficiente para calcular, mas **fome de dados** (*Data Starvation*). O gargalo é o barramento de memória (von Neumann bottleneck).
- **Compute-Bound:** hardware suficiente para os dados, mas **FPU's sobrecarregadas**. O gargalo são as unidades de execução.

6. Como o multithreading de hardware ajuda ou atrapalha

Memory-Bound

O multithreading **ajuda** enquanto há banda disponível:

- Múltiplos threads geram requisições independentes ao controlador de memória.
- O controlador pode reordenar e servir requisições em paralelo (interleaving entre bancos).
- Threads em núcleos diferentes exploram canais de memória distintos simultaneamente.

O multithreading **para de ajudar** quando:

- O barramento de memória está saturado — mais threads apenas aumentam contenção.
- Threads vão para núcleos com menor capacidade de emissão de requisições.

Compute-Bound

O multithreading **ajuda** com núcleos homogêneos:

- Cada núcleo tem sua própria FPU — paralelismo real de cálculo.
- Escala próxima do linear enquanto todos os cores são equivalentes.

O multithreading **atrapalha** quando:

- Threads são distribuídos entre núcleos com desempenho diferente, causando desbalanceamento de carga.
- O número de threads ultrapassa os núcleos físicos e threads lógicos passam a compartilhar unidades de execução (SMT/Hyperthreading), gerando contenção na FPU.
- O overhead de criação/sincronização de threads supera o ganho computacional.

7. Métricas recomendadas

Programa	Métrica principal	Justificativa
Memory-Bound	Largura de banda (GB/s)	O gargalo é o barramento. GFLOPS seria enganoso — o cálculo A+B é trivial; o que importa é quanto dado flui por segundo.
Compute-Bound	GFLOPS	O gargalo é a capacidade de cálculo. Mede diretamente a taxa de operações de ponto flutuante realizadas.
Ambos	Speedup $S = T_1/T_n$	Mede o ganho bruto de desempenho ao adicionar threads.
Ambos	Eficiência $E = S/n \times 100\%$	Mede o retorno marginal de cada thread adicional — revela o ponto de saturação e o custo de escalabilidade.

8. Conclusão

Aspecto	Memory-Bound	Compute-Bound
Gargalo	Barramento de memória	Unidades FPU/ALU
Melhor métrica	GB/s	GFLOPS
Escala com mais threads	Boa (até saturar o bus — caminho físico entre CPU e RAM)	Quase linear (até desbalancear)
Quando piora	Saturação do barramento	Desbalanceamento de carga
SMT/Hyperthreading	Pouco ganho	Pode piorar por contenção

Código

memory_bound.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define N 100000000

int main(int argc, char *argv[]) {
    int num_threads = 1;
    if (argc > 1) {
        num_threads = atoi(argv[1]);
        omp_set_num_threads(num_threads);
    }

    double *A = (double*)malloc(N * sizeof(double));
    double *B = (double*)malloc(N * sizeof(double));
    double *C = (double*)malloc(N * sizeof(double));

    if (!A || !B || !C) {
        fprintf(stderr, "Erro: falha na alocação de memória\n");
        return 1;
    }

    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        A[i] = 1.0;
        B[i] = 2.0;
    }

    double start = omp_get_wtime();

    #pragma omp parallel for
    for (long i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }

    double elapsed = omp_get_wtime() - start;

    double bandwidth_gbs = (3.0 * N * sizeof(double)) / elapsed / 1e9;

    int actual_threads = omp_get_max_threads();
    printf("RESULT threads=%d time=%.6f bandwidth_gbs=%.3f\n",
        actual_threads, elapsed, bandwidth_gbs);

    free(A); free(B); free(C);
    return 0;
}
```

compute_bound.c

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

#define N      50000 /* iteracoes externas */
#define INNER  10000 /* iteracoes internas de calculo intensivo */

int main(int argc, char *argv[]) {
    int num_threads = 1;
    if (argc > 1) {
        num_threads = atoi(argv[1]);
        omp_set_num_threads(num_threads);
    }

    double *resultado = (double*)malloc(N * sizeof(double));
    if (!resultado) {
        fprintf(stderr, "Erro: falha na alocao de memoria\n");
        return 1;
    }

    double start = omp_get_wtime();

    #pragma omp parallel for schedule(static)
    for (int i = 0; i < N; i++) {
        double temp = (double)i;
        for (int j = 0; j < INNER; j++) {
            temp = sin(temp) + cos(temp);
        }
        resultado[i] = temp;
    }

    double elapsed = omp_get_wtime() - start;

    double gflops = ((double)N * INNER * 2.0) / elapsed / 1e9;

    int actual_threads = omp_get_max_threads();
    printf("RESULT threads=%d time=%.6f gflops=%.3f\n",
        actual_threads, elapsed, gflops);

    free(resultado);
    return 0;
}
```